POLITECNICO
MILANO 1863

SCHOOL OF INDUSTRIAL AND
INFORMATION ENGINEERING

RASD

Best Bike Paths

Fabio Vokrri (10738386)
Soufian Sadgal (11003465)

December 23, 2025
Version 1.0

Contents

List of Figures

List of Tables

Listings

1 Introduction

1.1 Purpose

The *Best Bike Paths* (*BBP*) system has one objective in mind: support the ever-growing cycling community by providing a platform for recording, browsing, and sharing optimal bike routes. By integrating automated data acquisition from mobile sensors with manual user contributions, BBP builds a reliable inventory of safe and smooth paths. It enhances the biking experience by offering detailed performance statistics and real-time meteorological information on the go. Furthermore, BBP promotes safety by automatically identifying physical road hazards through community-verified data, which non-registered users can also benefit from. By prioritizing routes where speed limits and traffic density are bike-friendly, it promotes a safer environment for riders of all skill levels. Through its dual-mode insertion and analytical scoring, BBP becomes a comprehensive tool for both personal fitness and community navigation.

1.1.1 Goals

- G1: Registered Users are able to keep track of their cycling activities over time.
- G2: Information about bike paths is continuously enriched based on registered users' cycling activities and reports.
- G3: Automatic reports provide validated data about trips.
- G4: Registered users are provided with statistics about their cycling performance.
- G5: Trip data is enriched with meteorological information.
- G6: Bike path information is accessible to both registered and unregistered users.
- G7: Users are supported in selecting suitable bike paths

1.2 Scope

The core function of Best Bike Paths (BBP) is to collect, integrate, and provide reliable information about bike paths in order to support cyclists in making informed decisions about the routes they choose to follow. The system acts as a mediator between real-world bike path conditions, cyclists' experiences and reports, and external data sources such as meteorological services. Through BBP, users can contribute with information about bike path conditions, validate data automatically collected by the system, and explore available routes based on their suitability and current conditions. Registered users can record their trips, delete old trips, manually or automatically report obstacles or unsafe situations, confirm or correct detected information, and publish validated data to the community. Unregistered users can access publicly available information but are not allowed to contribute or modify data. The system interacts with external weather services to retrieve environmental data and enrich trip information. It also integrates with mobile devices to receive sensor data, such as GPS traces and accelerometer readings,that may indicate relevant path features or anomalies. BBP maintains a consolidated and continuously updated view of bike path conditions by merging contributions from multiple users according to their freshness, reliability, and level of confirmation.

1.2.1 World Phenomena

- WP1: Users physically drive their bike from one location to another over a period of time.
- WP2: Mobile devices record changes of their sensors, like GPS coordinates [1], orientation, speed and acceleration, independently from the system.
- WP3: Weather conditions, temperature and wind intensity change in a specific location over a period of time.
- WP4: Road conditions change in a specific location over a period of time.
- WP5: Users encounter obstacles along their way.
- WP6: Users ride their bikes in trafficked roads.
- WP7: Users develop opinions over certain paths.

1.2.2 Shared Phenomena

World Controlled Shared Phenomena

- SP1: Users register to the service via a specific form.
- SP2: Users log in via a specific form.
- SP3: External services provide weather data to the system in response to a request, if available.
- SP4: Registered users manually insert bike path status information.
- SP5: Registered users correct automatically acquired data about the bike path status.
- SP6: Registered users confirm automatically acquired data about the bike path status.
- SP7: Registered users publish information about bike paths collected manually or automatically.
- SP8: Users request information about available bike paths.
- SP9: Registered users plan a biking trip, causing the trip to be stored and managed by the system.
- SP10: Registered users accept the automatic tracking of the system.
- SP11: Registered users' devices share GPS data and other relevant sensor measurements to the system.
- SP12: Registered users update their profile information.
- SP13: Registered users delete their account from the system.
- SP14: Registered users delete trips from their trip history.
- SP15: Registered users access to their stored trip history.
- SP16: Users specify departure and destination points for route planning.

[1] GPS coordinates are registered by the device only if the user enables it. We assume that the device location is always on.

Machine Controlled Shared Phenomena

- SP17: The system provides suggested bike paths between the selected departure and destination points.
- SP18: The system sends requests to an external service to query weather information.
- SP19: The system sends requests to an external service to query map data.
- SP20: The system informs users about the presence of obstacles along a path associated with a planned trip.
- SP21: The system informs users about adverse weather in the location of the planned trip.

1.3 Definitions, Acronyms, Abbreviations

1.3.1 Definitions

- Software: ordered set of instructions that tells the computer how to use the available data to perform the tasks it is designed to carry out.
- Biking Trip: journey performed by bike over a period of time following a predetermined route.
- External Service: software outside the application's domain, consumed and relied upon by the primary application in order to perform the actions it is designed to execute.
- Obstacle: something, possibly dangerous, that hinders the biking trip or might block the route. Some examples of obstacles may be potholes, tree branches on the route, roadworks or road closures.
- Accelerometer: electronic sensor that measures the acceleration of the device.
- Gyroscope: electronic sensor that measures the orientation and angular velocity of the device.
- GPS (Global Positioning System): system using satellite orbiting the earth to obtain the device's location.

1.3.2 Acronyms

- BBP: Best Bike Paths.
- GPS: Global Positioning System.
- UI: User Interface.

1.3.3 Abbreviations

- G: goal.
- WP: World Phenomena.
- SP: Shared Phenomena.

1.4 Revision history

- Version 1.0: 23/12/2025

1.5 Reference Documents

The assignment for this document and all the information included refer to the following documentation:

- The specification for the 2025/26 Requirement Engineering and Design Project for the Software Engineering II course.
- The slides on the WeBeep page of the Software Engineering II course.

1.6 Document Structure

1. Introduction: a brief description of the project that follows from the analysis of the specification provided, it highlights the goals of the product and its scope.
2. Overall Description: a more detailed description of the product that comprehends also real world scenarios and analyses the system from a technical point of view.
3. Specific Requirements: a detailed list of all the requirements needed in order to satisfy the goals, and their relationship with domain assumptions.
4. Formal analysis using Alloy: a formal description of a portion of real world phenomena analysed through Alloy in order to highlight inconsistencies between assumptions and phenomena previously modelled.
5. Effort spent: the time spent by each member of the group to contribute to the realization of this document.
6. References: reference to documents or tools used for writing this document.

2 Overall Description

2.1 Product Perspective

2.1.1 Scenarios

User Registration Non-registered user U accesses the system and decides to create an account. U provides the required credentials (email and password) and completes the registration process.

Trip Planning and Path Selection User U (registered or not) specifies a departure and a destination point in order to plan a bike trip. The system displays the available bike paths, ranked by their current condition, and weather information, if available. U compares the options and selects the most suitable path for his trip.

Review of Trip Statistics After completing a cycling journey, registered user U reviews the statistics associated with the performed trip. The system displays information such as distance, duration, and average speed, allowing the user to analyse the outcome of the journey.

Validation of Automatically Acquired Data During a bike trip, the system automatically collects information about the bike path based on sensor data. Registered user U reviews the collected information, corrects potential inaccuracies, and confirms the report. Once validated, U publishes the report to make it available to the community.

Manual Reporting After Discarding Automatic Data After completing a bike trip, registered user U decides that the automatically collected data does not accurately reflect the perceived path conditions. U discards the automatically acquired data and manually inserts information about the bike path. The manually created report is then published.

Attempted Publication Without Validation Registered user U attempts to publish automatically collected information without validating it. The system prevents the publication and notifies U that validation is required and it is then prompted to correct and validate the data before publishing it.

Notification of Changes and Replanning After planning a bike trip, registered user U receives a notification indicating changes affecting the selected path, such as adverse weather conditions or the presence of a newly reported obstacle. Based on this information, U decides to cancel the planned trip and proceeds to plan an alternative route.

Account Deletion Registered user U accesses the account management section and requests the deletion of their account. The system processes the request and removes his account.

Trip Deletion Registered user U decides to delete a completed trip from his cycling activities. The trip, including all statistics, is deleted from the user's history, but the underlying information about the path remains untouched, so other users can take advantage of the data when planning their trip.

2.1.2 Domain Class Diagrams

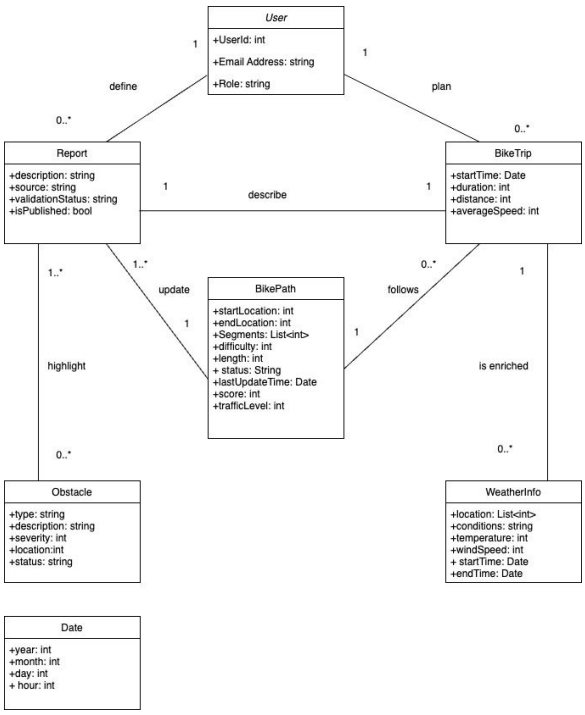


Figure 1: Domain Class Diagram

The class diagram represents a domain-level conceptual model of the BBP system. It captures the main entities of the application domain and their relationships, focusing on what information is managed rather than how it is technically implemented. External systems, devices, and technical components are intentionally excluded, while primitive types and generic collections are used to represent abstract data without imposing design-level constraints. The choice to model Report, Obstacle, and WeatherInfo as independent classes rather than simple attributes is motivated by their intrinsic complexity and lifecycle. Reports represent structured contributions that can be manually or automatically generated, validated, and published, and therefore require their own attributes and state. Obstacles are modeled as separate entities because they have specific properties such as severity, location, and resolution status, and may persist independently of individual reports. Weather information is represented as a class because weather conditions are time-dependent and can vary during a trip, making it unsuitable to model them as static attributes of a bike path.

2.1.3 State Diagrams

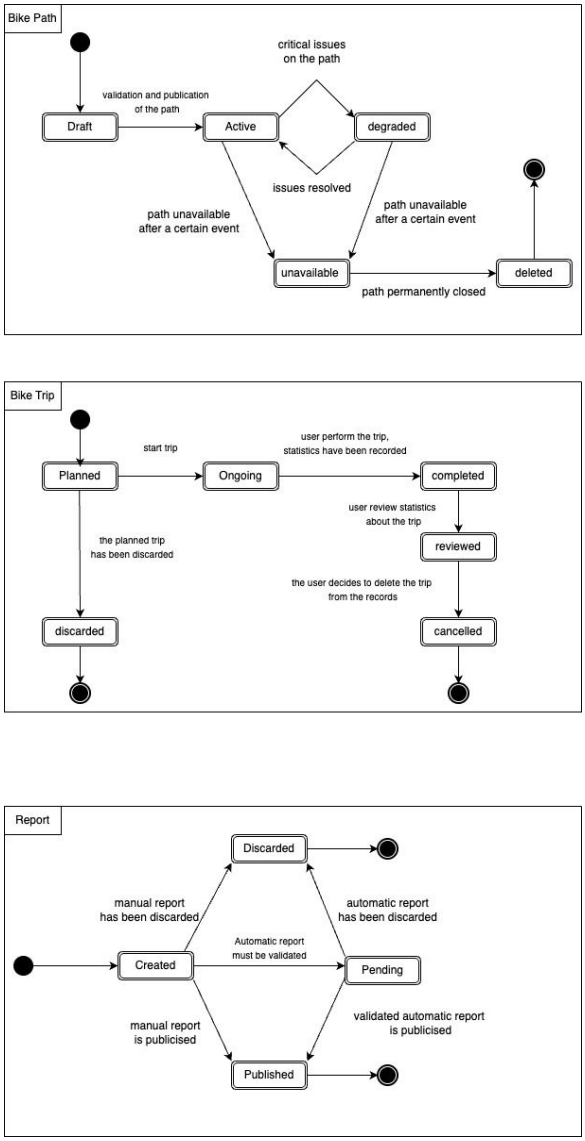


Figure 2: State Diagrams.

Bike Path State Diagram The state diagram models the lifecycle of a bike path within the BBP system, describing how the status of a path evolves as information is created, validated, updated, and affected by reported conditions or external events. The concept of critical issues that causes a bike path to transition from the Active state to the Degraded state is defined by the presence of obstacles associated with that path. In particular, each obstacle is characterised by a severity attribute, and the overall status of the path is determined by aggregating the severity levels of the obstacles reported by users along the path. When one or more obstacles exceed a predefined severity

threshold, the path is considered degraded.

Bike Trip State Diagram This state diagram models the lifecycle of a bike trip as an information object managed by the system, not merely the physical activity of cycling. The distinction between Discarded and Cancelled is conceptual: Discarded refers to trips abandoned before execution and therefore never contributing to historical analytics, while Cancelled refers to trips that were completed and reviewed but are later explicitly removed from the user's records, without affecting aggregated or anonymised system-level statistics.

Report State Diagram The state diagram models the life cycle of a Report, capturing both manually created and automatically generated reports within a single behavioural model. Although manual and automatic reports follow partially different validation paths, they share the same core states and final outcomes, namely creation, publication, or discard. Manual reports are created directly by users and can be published immediately after creation, as their content is explicitly provided and confirmed by the user at the time of insertion. For this reason, manual reports can transition directly from the Created state to the Published state or be discarded without undergoing additional validation steps. Automatic reports, instead, are generated by the system based on sensor data collected during a bike trip. Since such data may be inaccurate or misinterpreted, automatic reports must always pass through the Pending state, in which users are required to validate or correct the information before it can be published. Only after successful validation can an automatic report transition to the Published state; otherwise, it may be discarded.

2.2 Product Functions

User account management The system provides user account management functionalities that allow individuals to register and access the platform as registered users, to delete the account and to update their profile information. This enables personalised features such as trip recording, reporting, and notifications, while still allowing non-registered users to access publicly available information.

Trip recording and performance statistics The system allows registered users to record their cycling trips. Collected data is stored to build a history of cycling activities, enabling users to review performance statistics such as distance, duration, and average speed, and to analyse their progress over time. The system also allows users to manage their trip history by deleting individual past trips, so that personal records and aggregated performance metrics are computed based on non-deleted trips only.

Path browsing and search The system provides an inventory of bike paths that can be browsed and searched by both registered and non-registered users. Publicly available information about paths, including their status and overall conditions, is made accessible to support route exploration and discovery.

Route planning and path recommendation The system supports users in planning their trips by allowing them to specify a departure and a destination. Based on the available information, the system returns a set of possible bike paths, ordered according to their overall score.

Path reporting The system enables users to contribute information about bike paths by reporting conditions, obstacles, and other relevant details after completing a trip. Reports can be created manually or derived automatically from sensor data collected during the trip, contributing to a richer and more detailed description of bike paths.

Report validation and publication The system allows users to review, confirm, or correct automatically collected information before it is made publicly available. Only validated reports are published, ensuring that shared information about bike paths is reliable and consistent.

User notifications on relevant updates The system provides notifications to users when relevant changes occur, such as updates in weather conditions or the presence of newly reported obstacles affecting planned trips. This allows users to adapt their plans and make informed decisions in a timely manner.

2.3 User Characteristics

2.3.1 Registered Users

Users are able to register and login to the platform. The registered user's profile contains his personal trips and cycling activities, including statistics about each trip, such as total distance covered, average speed and other performance metrics. Registered users can insert and make publishable information about bike paths, including their status (e.g. optimal, medium, sufficient, requires maintenance) and the presence of relevant obstacles, like potholes. Registered users can manually or automatically insert information about paths:

- Manual mode: users manually insert data, specifying the name and the status of each street they cycled through during the trip.
- Automated Mode: users let the system acquire data from their mobile device while they bike. They must then confirm or correct the information acquired by the system before making it available to the community.

Registered users are able to share their location information and device sensors data to the system in order to automatically acquire information about the bike path they are cycling through. Also, the data they are sharing to the system is used to notify them of possible obstacles along their way during the cycling session, inserted by other users. Lastly, users are able to accept or deny the BBP system to gather weather information from external services through the device's internet connection. Registered users can specify a departure and destination point and ask the system to visualize on a virtual map all the bike paths connecting these two points, ranked by a score computed based on the status of the path and how effective it is to let the user reach the destination point. Registered users are able to change their preferences about GPS location sharing and internet access. For instance, if they previously granted GPS location sharing to the BBP system and don't want to share their location anymore, they can deny it. Registered Users are able to modify and delete their cycling activities. Registered users are able to delete their accounts and all personal data associated with it.

2.3.2 Non-Registered Users

Non-registered users can also take advantage of the data collected by the BBP system. Non-registered users can specify a departure and destination point and ask the system to visualize on a virtual map

all the bike paths connecting these two points, ranked by a score computed based on the status of the path and how effective it is to let the user reach the destination point.

2.4 Assumptions, Dependencies and Constraints

2.4.1 Domain Assumptions

- D1: Users aim to provide truthful and accurate information when reporting bike path conditions
- D2: Users' devices have accelerometer and gyroscope sensors through which the BBP system automatically gathers information about biking paths.
- D3: Data gathered from devices' sensors is accurate.
- D4: Users consent to enable and share their GPS location to the BBP [2] specify that otherwise some main functionalities of the BBP system will not be available
- D5: Users' devices have internet connectivity when accessing BBP services.
- D6: The external map service reliably responds with geographical data when the BBP system sends the request.
- D7: The geographical information gathered from the external map service is accurate.
- D8: The external weather service reliably responds with meteorological data when the BBP system sends the request.
- D9: The meteorological information gathered from the external weather service is accurate.
- D10: A sufficient number of users actively contribute reports in order to keep bike path information up to date.

2.4.2 Dependencies

1. External Map Service: BBP system relies on an external virtual map provider (i.e. Google Maps, MapBox or OpenStreetMap) to gather information about the users' trip (mapping between current users positions and street names) and to allow them to visualize best biking paths connecting the departure and destination points.
2. External Weather Service: BBP system relies on an external meteorological data provider (i.e. OpenWeatherMap, AccuWeather) to gather information about the wind speed, temperature and humidity in users' current location.
3. Mobile Operating System: BBP system relies on operating systems frameworks (i.e. iOS, Android) to access low-level hardware sensors, as accelerometer, gyroscope and GPS [3]

[2] Otherwise main functionalities of the BBP System won't be available.

[3] We assume that the GPS location is on by default and that the operating system of the device shares the location information under users' permission. We could have modelled the software so that it directly interacted with and relied upon the GPS framework independently of the operating system in use, but real life applications rely on the underlying operating system to collect users' location information. Therefore, we treat the GPS signal as a sensor of the device.

2.4.3 Constraints

1. Network Connectivity: The system shall restrict access to external data and report publication when network connectivity is not available.
2. Data collection: The system must not collect or process location data without explicit user consent.
3. Privacy: The collection of location data must comply with data protection regulations (e.g. GDPR).

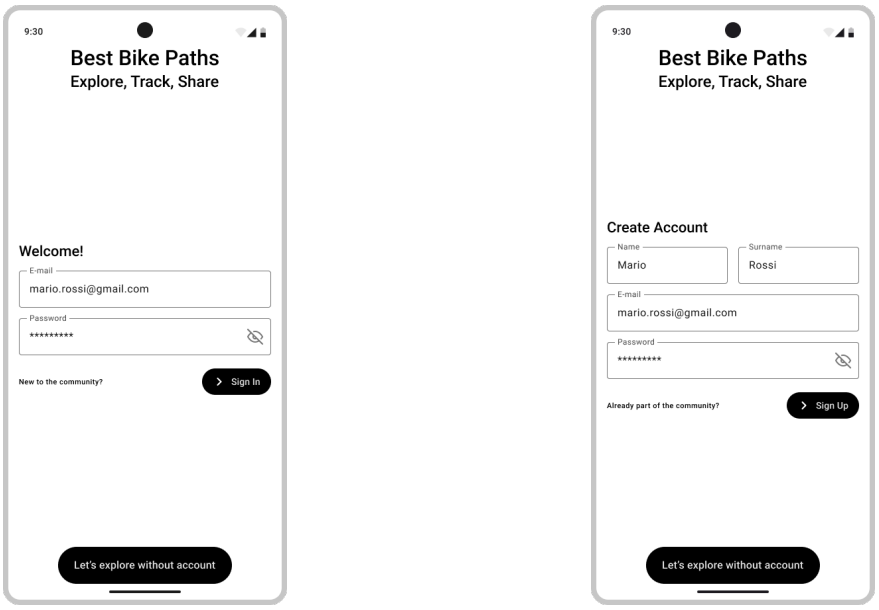
3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The following figures show a mock-up user interface of the mobile application for the Best Bike Paths system. The images exhibit the most important screens from the perspective of a registered user. The home screen shown in figure 4 and the profile screen shown in figure 7, are replaced by a direct link to the registration page if the user did access the application without signing up.

Sign In and Sign Up Pages In this screen the user can sign in by entering e-mail and password, if already registered. If the user is new to the platform, can click the "*New to the community?*" button and access the sign up screen, where he can enter his name, surname, email and password to create an account. If a registered user accidentally entered the sign up screen, can go back to the sign in screen by clicking the "*Already part of the community*" button. In both screens, non-registered users can access the map screen shown in figure 5 by clicking the "*Let's explore without account*" button.



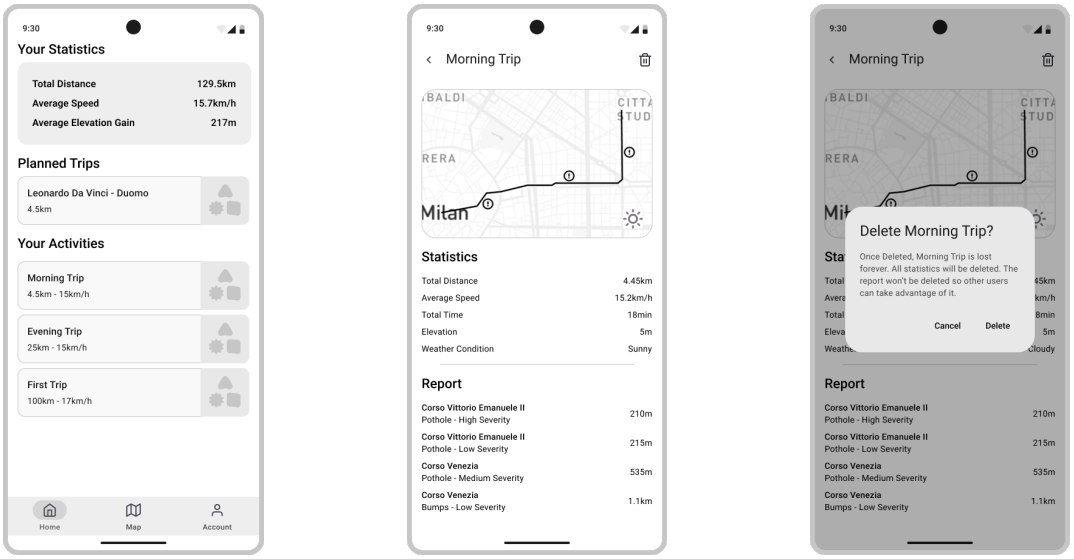
(a) Sign In Screen.

(b) Sign Up Screen.

Figure 3: Sign in and Sign up screens.

Home Screen In this screen the registered user can visualize his statistics, the trips he planned and all his biking activities. By clicking on a saved trip, the user can immediately start the navigation of the selected trip, as shown in figure 5d. By clicking on an activity, the user can see the details regarding that specific activity, like total distance, average speed, total time, elevation and weather conditions, and also the report of all obstacles encountered during his trip. In this screen the user can also delete the activity by clicking on the trash button (top right corner), after confirmation, or

go back to the home page by clicking the back button (top left corner). This screen is reserved to registered users only.



(a) Home Screen. (b) Sign Up Screen. (c) Deletion of an activity.

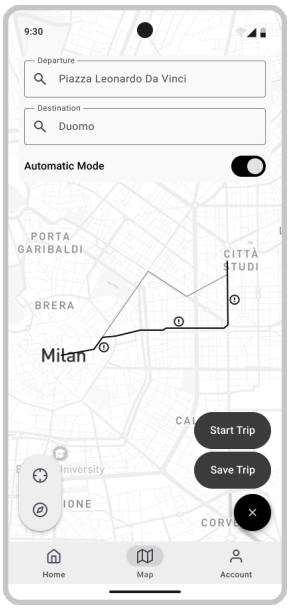
Figure 4: Home Screen and Trip Details Screen.

Map Screen In this screen users can browse the map, visualize their current position on the map, by clicking the first button of the floating menu on the bottom left corner, and change the map orientation by clicking to the second button of the floating menu. By inserting a departure and destination point in the first and second input fields respectively, the user can see on the map different paths that lead from the departure point to the destination one. By clicking on a path, the user can see the major obstacles along the way signalled by other users and the score of every segment of the path coded in different colours.

The following is reserved to registered users only. When clicking on a path, a switch appears under the input fields, allowing the user to activate automatic data acquisition when cycling. Also a floating action button on the bottom right corner appears, so the user can start biking the selected route. Once clicked, the floating action button reveals two options: "*Save Trip*" or "*Start Trip*", which let the user choose between saving the trip for later cycling sessions, or immediately starting the trip. By pressing the "*Save Trip*" button, the user can visualize his trip in the home screen, shown in figure 4a. By pressing the "*Start Trip*", the user is taken to the navigation page, where he can stop the recording at any time by clicking on the "*Stop Recording*" button, add a new obstacle on his way by clicking the bottom right floating action button with the plus sign. or switch between manual or automatic mode, by clicking the switch in the bottom left corner.

(a) Map Screen.

(b) Map Screen Trip Planning.



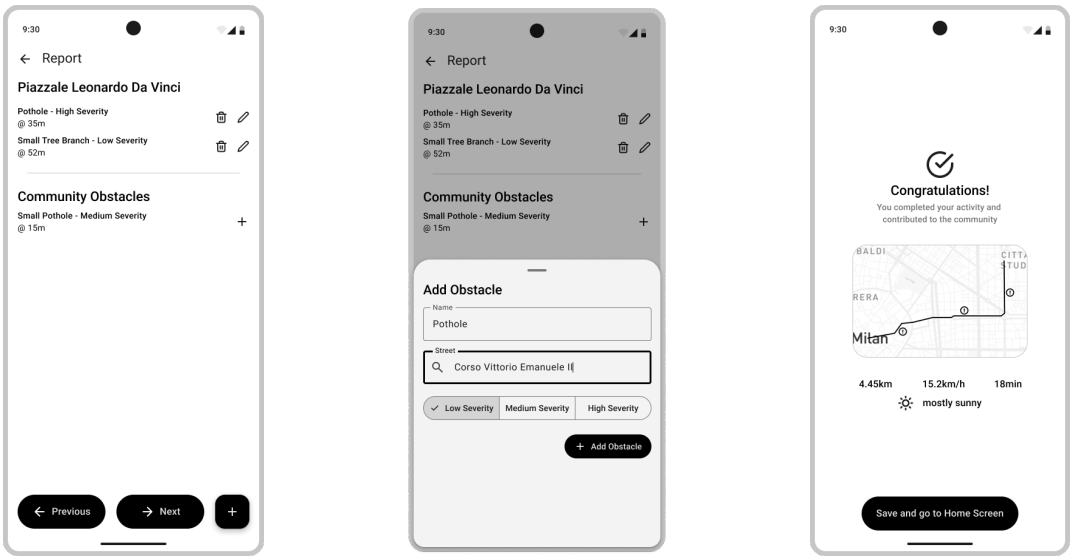
(c) Map Screen Trip Selection.

(d) Map Navigation.

Figure 5: Map Screens and Navigation.

Report Screen This screen is presented to registered users who terminated their biking session. Here can manually add obstacles found along the way, or review the ones added with the floating action button with the plus sign in the bottom right corner found in the previous screen, shown in figure 5d. Here can manually modify the name of the obstacle, the street where the obstacle

was found and the severity of the obstacle. Also, the registered user is suggested with obstacles signalled by other members of the community, so it's easier for him to insert obstacles. The user can also delete obstacles he accidentally inserted, by clicking in the trash icon button next to the name of the obstacle. The same set of operations can be performed by a registered user who chose to automatically gather information when presented with the floating action button option in screen 5c. This screen is presented to the user for every street he went through during his journey. He can go the next route by pressing the "*Next*" button or go to the previous one by clicking on the "*Previous*" button. Once the review of the report is completed, the user is presented with a greeting screen, where he can visualize his statistics and go to the home screen. This screen is reserved to registered users only.



(a) Report Screen. (b) Add obstacle. (c) Congratulations Screen.

Figure 6: Report and Congratulation Screens.

Profile Screen In this screen the user can see his personal information and the number of activities and contributions he has made. Also, he can change his preferences via the toggle button or delete his account via the trash button. This screen is reserved to registered users only.

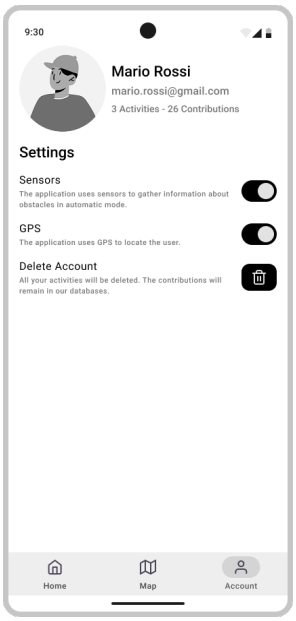


Figure 7: Profile Screen.

3.1.2 Hardware Interfaces

The BBP system is a mobile application, requiring only a device with Android or iOS operating system installed. The device must have integrated sensors, GPS and internet access in order to allow the software to properly run.

3.1.3 Software Interfaces

The BBP system relies on the following software interfaces:

- Weather API: the acquired data about the biking trip is enriched with weather information gathered from an external service.
- Map API: the BBP software must allow the user to visualize his current position in a virtual map, to show on the map different biking trips connecting the departure to the destination point, and allow the user to use the map to navigate through his journey. The application relies on an external map provider to gather this geographical information.

3.1.4 Communication Interfaces

The BBP system as well as gathering external weather and map information, must also query all the needed data about roads, obstacles and registered users from the centralized database. The communication with the back-end must be secure and encrypted, especially for sensitive user data, such as name, surname email and password, so the HTTPS protocol is used. The data is then transferred using the JSON format.

3.2 Functional Requirements

Account and Access Management

- R1: The system shall allow biker to create an account.
- R2: The system shall allow registered users to log in.
- R3: The system shall allow registered users to update information in their profile.
- R4: The system shall allow registered users to delete their account.

Bike Path Discovery and Visualisation

- R5: The system shall allow users to browse the inventory of paths,
- R6: The system shall allow users to visualise available bike paths on a map provided by an external map service.

Trip Planning and Route Recommendation

- R7: The system shall allow registered users to plan a bike trip by storing it among planned trips,
- R8: The system shall allow the user to specify a starting and ending point in order to retrieve all the possible path between them.
- R9: In case a registered user has provided a departure and a destination point, the system shall return the possible paths ordered based on their scores.

Trip Tracking, History and Performance Analytics

- R10: The system shall allow registered users to record their trips.
- R11: The system shall provide registered users with statistics about their cycling performance.
- R12: The system shall allow registered users to view all the non deleted past trips.
- R13: The system shall allow registered users to delete trips from their trip history.
- R14: The system shall allow registered users to enable or disable automatic trip tracking.

Reporting, Validation and Publication of Path Information

- R15: The system shall allow registered users to manually produce a report.
- R16: The system shall allow registered users to automatically produce a report.
- R17: The system shall allow registered users to review and validate reports produced automatically.
- R18: The system shall allow registered users to publish manual reports.
- R19: The system shall allow registered users to publish validated reports produced automatically.

Context Enrichment and Notifications

- R20: The system shall notify registered users with planned trips when relevant weather conditions affecting the trip exceed a predefined severity threshold [4].
- R21: The system shall notify registered users with planned trips when obstacles along the associated path exceed a predefined severity threshold [5].
- R22: The system shall enrich trips in the inventory with meteorological information retrieved from an external weather service when available.

[4]The system alerts the registered user of a relevant weather condition along a planned trip only when the external weather service provides an adverse forecast.

[5]The system alerts the registered user of a given obstacle along a planned trip only when other registered users report an obstacle with a high severity (a combination of temperature, wind speed and humidity).

3.2.1 Use Case Diagram

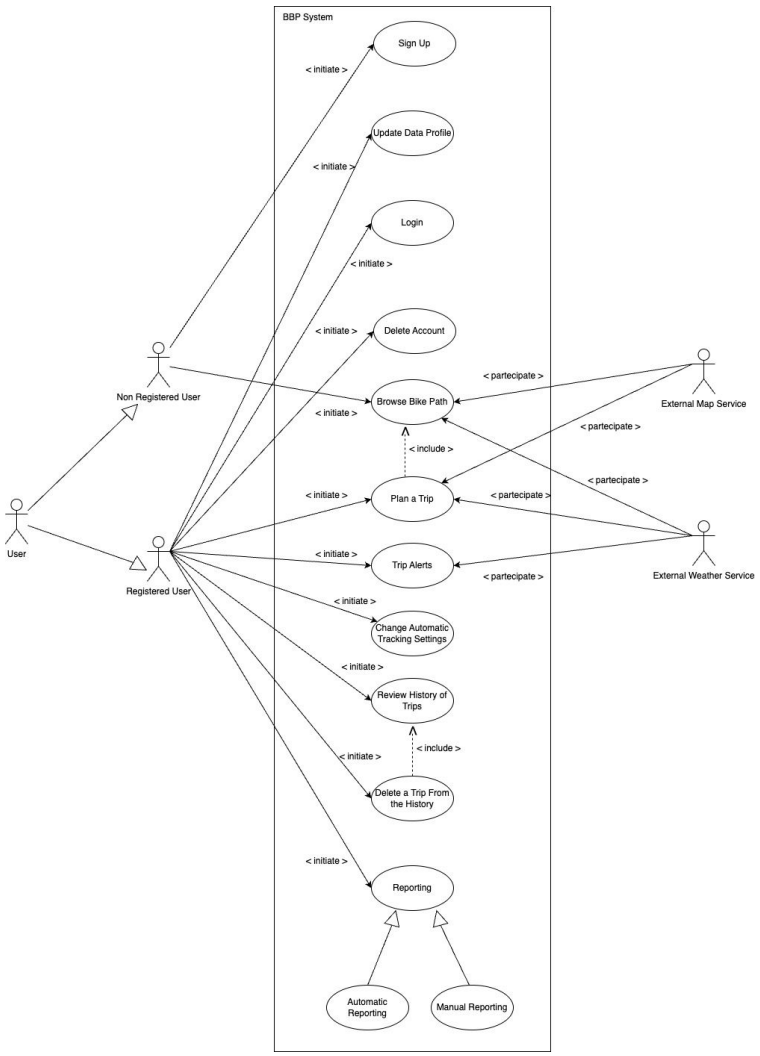


Figure 8: Use Case Diagram.

3.2.2 Use Cases

UC1: User Registration

Name	**User Registration**
Actors	Non-Registered User.
Entry Condition	The user has access to the BBP system.
Event Flows	• The user requests to create an account. • The system requests registration data. • The user provides registration data. • The system validates the data and creates the account.
Exit Condition	The user successfully registered to the platform.
Exception	• Account already exists with associated email. • Provided data is invalid.
Special Requirements	Privacy: GDPR compliance for personal data handling, password storage

Table 1: UC1: User Registration

UC2: User Login

Name	**User Login**
Actors	Registered User.
Entry Condition	A registered account exists.
Event Flows	• The user requests to log in. • The system requests credentials. • The user provides credentials. • The system authenticates the user and grants access.
Exit Condition	The user is authenticated and has access to registered features.
Exception	• Account not available (deleted/blocked). • Invalid credentials.
Special Requirements	Security: Authentication and session handling

Table 2: UC2: User Login

UC3: Update Profile Data

Name	**Update Profile Data**
Actors	Registered User.
Entry Condition	The user is authenticated.
Event Flows	• The user requests to update profile information. • The system displays current profile data (conceptually: makes them available for editing). • The user submits updated data. • The system validates and stores the updates.
Exit Condition	User's profile data is updated.
Exception	• Invalid new profile information.
Special Requirements	Privacy: GDPR compliance for personal data handling, password storage

Table 3: UC3: Update Profile Data

UC4: Delete Account

Name	**Delete Account**
Actors	Registered User.
Entry Condition	The user is authenticated.
Event Flows	• The user requests account deletion. • The system performs the deletion according to retention rules. • The system revokes access.
Exit Condition	The account is deleted and the user becomes non-registered.
Exception	
Special Requirements	Privacy: GDPR compliance for personal data handling, password storage

Table 4: UC4: Delete Account

UC5: Browse Bike Paths

Name	**Browse Bike Paths**
Actors	1. User (Registered or not). 2. External Map Service, External Weather Service.
Entry Condition	The user has access to the BBP system
Event Flows	• The user requests to explore available bike paths. • (Optional) The user provides departure–destination points. • The system retrieves the matching paths. • The system provides path information (status, score, obstacles/reports summary). • (Optional) The system retrieves and shows map representation via the external map service. • (Optional) The system retrieves and shows weather information for the relevant area/time.
Exit Condition	The user explores available paths and their information.
Exception	No paths match the request.
Special Requirements	Compatibility: interoperability with external services (map/weather systems).

Table 5: UC5: Browse Bike Paths

UC6: Plan Trips

Name	**Plan Trips**
Actors	1. Registered User. 2. External Map Service, External Weather Service.
Entry Condition	The user is authenticated.
Event Flows	• The user requests to plan a new trip. • The user selects a bike path • The system stores the trip in the planned trips list. • (Optional) The system enriches the planned trip with available weather information.
Exit Condition	A planned trip is stored and managed by the system.
Exception	• Selected path becomes unavailable during planning.
Special Requirements	Performance: time behaviour.

Table 6: UC6: Plan Trips

UC7: Change Automatic Tracking Settings

Name	**Change Automatic Tracking Settings**
Actors	1. Registered User. 2. User Device (sensors and GPS).
Entry Condition	The user is authenticated.
Event Flows	• The user requests to enable or disable automatic trip tracking. • The system stores the user's tracking preference. • (If enabling) the system requests to collect sensor/GPS data.
Exit Condition	Tracking preference is updated.
Exception	• User device does not support required sensors. • User denies required permissions/consent.
Special Requirements	Privacy: explicit consent for location/sensor tracking.

Table 7: UC7: Change Automatic Tracking Settings

UC8: View History and Statistics of Completed Trips

Name	**View History and Statistics of Completed Trips**
Actors	Registered User
Entry Condition	The user is authenticated.
Event Flows	• The user requests access to trip history. • The system retrieves all non-deleted completed trips. • The user selects a trip. • The system provides the trip details and related statistics.
Exit Condition	The user has reviewed past trips and their statistics
Exception	No trips exist in the history

Table 8: UC8: View History and Statistics of Completed Trips

UC9: Delete Trips from History

Name	**Delete Trips from History**
Actors	Registered User.
Entry Condition	The user is authenticated and at least one trip exists in history.
Event Flows	• The user requests deletion of a specific past trip. • The system removes the trip from the user's history (marks as deleted). • The system updates any aggregated views/metrics to exclude deleted trips.
Exit Condition	The selected trip is deleted from history.
Special Requirements	Data Consistency: deleted trips must not appear in global statistics and history views.

Table 9: UC9: Delete Trips from History

UC10: Manual Report

Name	**Manual Report**
Actors	Registered User.
Entry Condition	A trip has been completed.
Event Flows	• The user requests to create a manual report about a planned bike path. • The user provides report data (conditions, obstacles, notes) and request the publication • The system stores the report and publishes it as public path information.
Exit Condition	A manual report is published and contributes to path enrichment.
Exceptions	Report data is invalid or incomplete.

Table 10: UC10: Manual Report

UC11: Automatic Report

Name	**Automatic Report**
Actors	1. Registered User. 2. User's Device (data source).
Entry Condition	A trip has been recorded with automatic tracking enabled.
Event Flows	• The system generates an automatic report from collected trip data. • The user requests to review the generated report. • The user possibly corrects the report and confirm it • The system marks the report as validated • The user confirms the report and requests the publication • The system publishes the report.
Exit Condition	A validated automatic report is published.
Exceptions	• User discards the report: the report is not published.
Special requirements	Reliability: fault tolerance (incomplete data coming from device sensors).

Table 11: UC11: Automatic Report

UC12: Receive Trip Alerts

Name	**Receive Trip Alerts**
Actors	1. Registered User. 2. External Weather Service, Other (reporting) Users.
Entry Condition	At least one planned trip exists.
Event Flows	• The system detects relevant changes affecting a planned trip (e.g., adverse weather severity threshold exceeded; obstacle severity threshold exceeded). • The system notifies the user.
Exit Condition	The user receives the alert information
Exceptions	External weather service unavailable (weather alerts may be delayed/unavailable).

Table 12: UC12: Receive Trip Alerts

3.2.3 Sequence Diagrams

User Registration

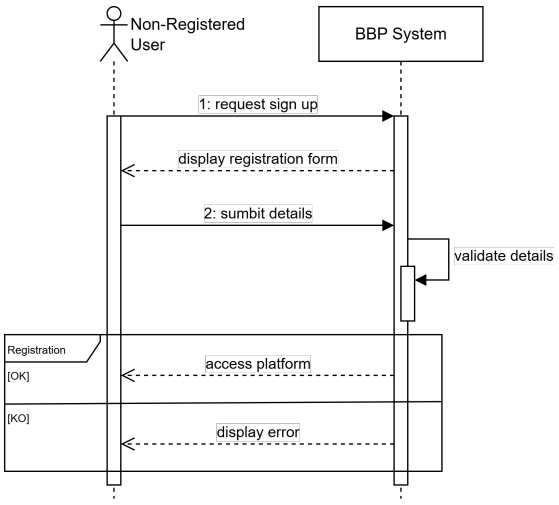


Figure 9: User Registration Sequence Diagram.

User Log In

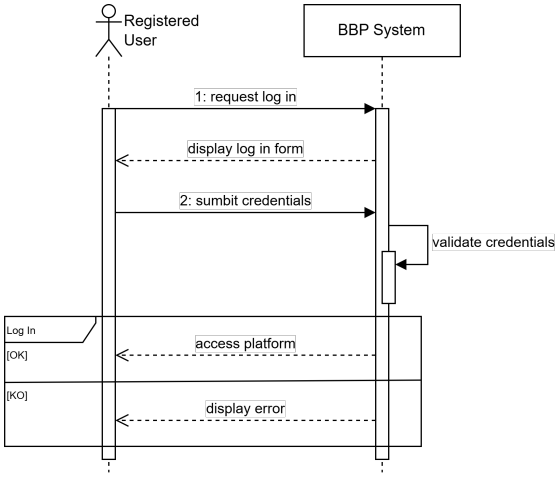


Figure 10: User Log In Sequence Diagram.

Update Profile Data

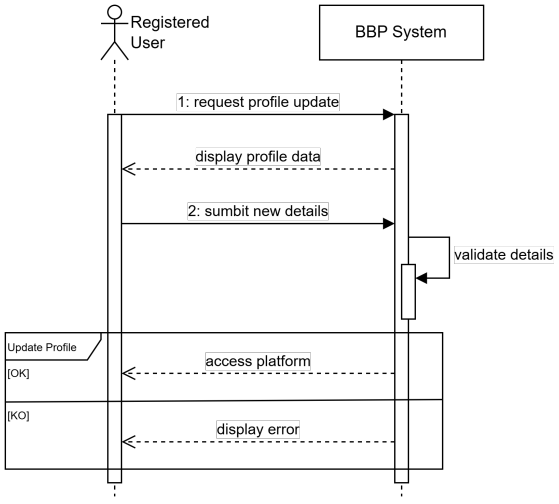


Figure 11: Update Profile Data Sequence Diagram.

Delete Account

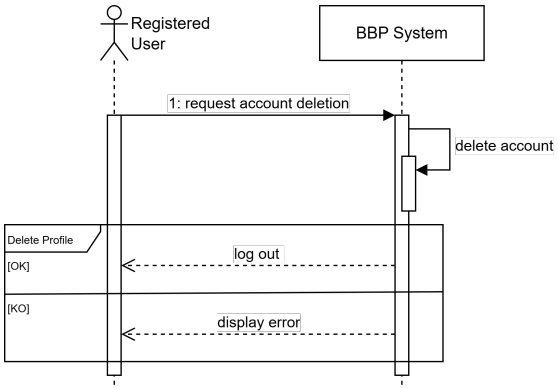


Figure 12: Delete Account Sequence Diagram.

Browse Bike Paths

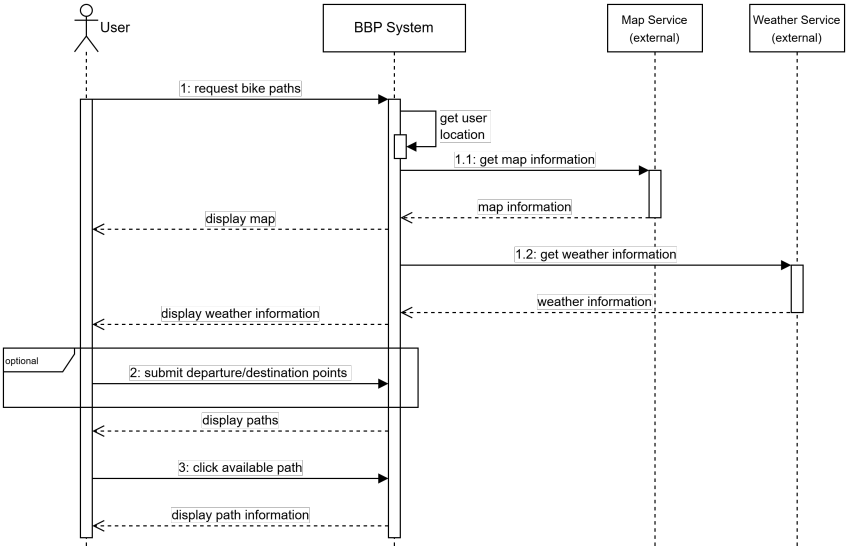


Figure 13: Browse Bike Paths Sequence Diagram.

Plan Trip

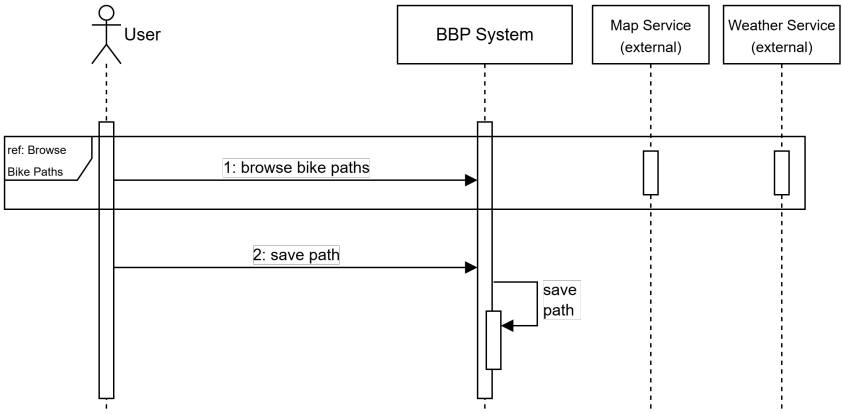


Figure 14: Plan Trip Sequence Diagram.

Change Sensor Preferences

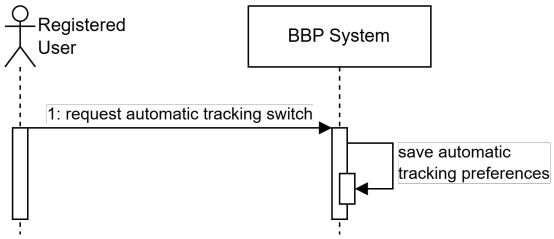


Figure 15: Change Sensor Preferences Sequence Diagram.

View Trip Details and Statistics

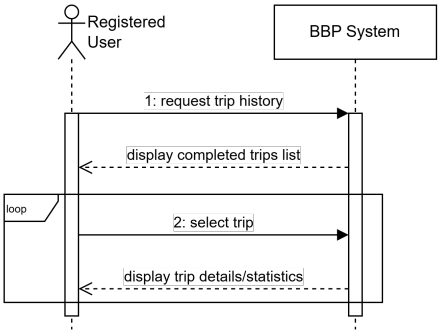


Figure 16: View Trip Details Sequence Diagram.

Delete Trip

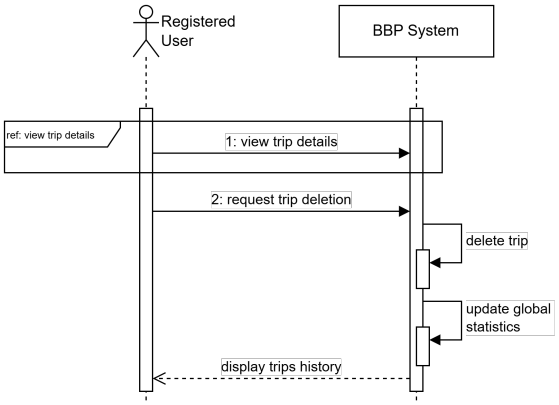


Figure 17: Delete Trip Sequence Diagram.

Manual Report

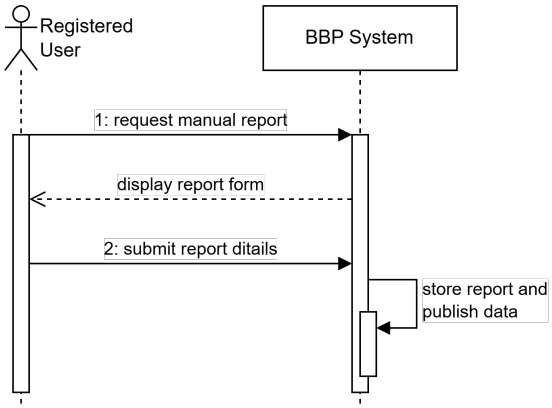


Figure 18: Manual Report Sequence Diagram.

Automatic Report

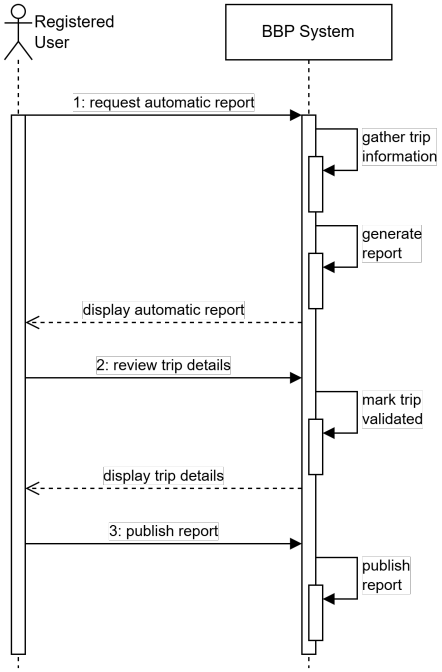


Figure 19: Manual Report Sequence Diagram.

Receive Trip Alerts

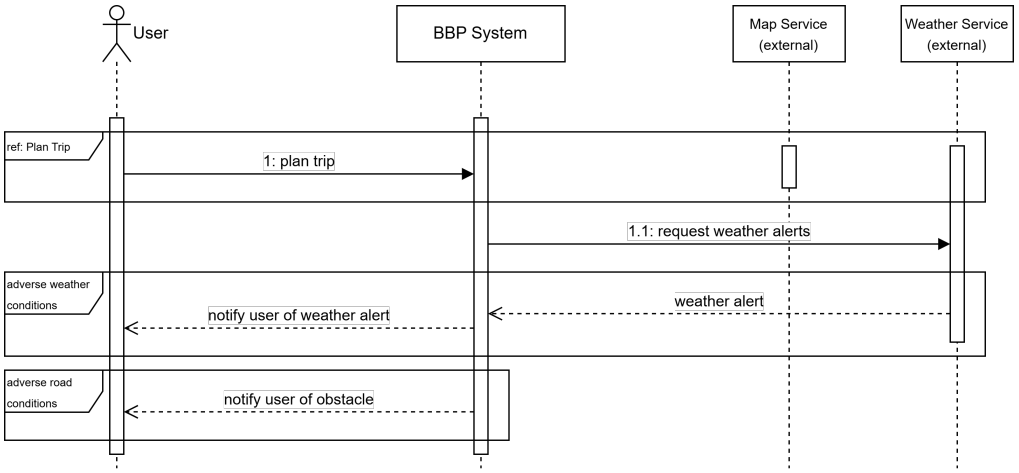


Figure 20: Receive Trip Alerts Sequence Diagram.

3.2.4 Mapping on Requirements

G1: Registered Users are able to keep track of their cycling activities over time.	
• R10: The system shall allow registered users to record their trips. • R11: The system shall provide registered users with statistics about their cycling performance. • R12: The system shall allow registered users to view all the non deleted past trips. • R13: The system shall allow registered users to delete trips from their trip history.	• D2: Users' devices have accelerometer and gyroscope sensors. • D3: Data gathered from devices' sensors is accurate. • D4: Users consent to enable and share their GPS location.

Table 13: Requirement Mapping of Goal G1.

G2: Information about bike paths is continuously enriched based on registered users' cycling activities and reports.	
• R14: The system shall allow registered users to enable or disable automatic trip tracking. • R15: The system shall allow registered users to manually produce a report. • R16: The system shall allow registered users to automatically produce a report. • R18: The system shall allow registered users to publish manual reports. • R19: The system shall allow registered users to publish validated reports produced automatically.	• D1: Users aim to provide truthful and accurate information when reporting. • D2: Users' devices have accelerometer and gyroscope sensors. • D3: Data gathered from devices' sensors is accurate. • D4: Users consent to enable and share their GPS location. • D10: A sufficient number of users actively contribute reports.

Table 14: Requirement Mapping of Goal G2.

G3: Automatically collected information about bike paths is validated by registered users before being made public.	
• R16: The system shall allow registered users to automatically produce a report. • R17: The system shall allow registered users to review and validate reports produced automatically. • R19: The system shall allow registered users to publish validated reports produced automatically.	• D1. Users aim to provide truthful and accurate information when reporting bike path conditions • D3. Data gathered from devices' sensors is accurate.

Table 15: Requirement Mapping of Goal G3.

G4: Registered users are provided with statistics about their cycling performance.	
• R11: The system shall provide registered users with statistics about their cycling performance. • R14: The system shall allow registered users to enable or disable automatic trip tracking.	• D2: Users' devices have accelerometer and gyroscope sensors. • D3: Data gathered from devices' sensors is accurate. • D4: Users consent to enable and share their GPS location.

Table 16: Requirement Mapping of Goal G4.

G5: Trip data is enriched with meteorological information.	
• R22: The system shall enrich recorded trips with meteorological information retrieved from an external weather service.	• D8: The external weather service reliably responds to requests. • D9: The meteorological information gathered is accurate.

Table 17: Requirement Mapping of Goal G5.

G6: Bike path information is accessible to both registered and unregistered users.	
• R5: The system shall allow users to browse the inventory of paths. • R6: The system shall allow users to visualise available bike paths on a map provided by an external map service. • R8: The system shall allow the user to specify a starting and ending point in order to retrieve all the possible path between them. • R9: In case a user has provided a departure and a destination point, the system shall return the possible paths ordered based on their scores.	• D6: The external map service reliably responds with geographical data. • D7: The geographical information gathered from the external map service is accurate.

Table 18: Requirement Mapping of Goal G6.

G7: Users are supported in selecting suitable bike paths.	
• R5: The system shall allow users to browse the inventory of paths. • R6: The system shall allow users to visualise available bike paths on a map. • R8: The system shall allow the user to specify a starting and ending point to retrieve possible paths. • R9: The system shall return the possible paths ordered based on their scores. • R20: The system shall notify registered users with planned trips when relevant weather conditions affecting the trip exceed a predefined severity threshold. • R21: The system shall notify registered users with planned trips when obstacles along the associated path exceed a predefined severity threshold. • R22: The system shall enrich recorded trips with meteorological information retrieved from an external weather service	• D6: The external map service reliably responds with geographical data. • D7: The geographical information gathered from the external map service is accurate. • D8. The external weather service reliably responds with meteorological data when the BBP system sends the request. • D9. The meteorological information gathered from the external weather service is accurate.

Table 19: Requirement Mapping of Goal G7.

3.3 Performance Requirements

The BBP system must ensure that the collected information is available both to registered and non-registered users. Also, the system performs well only if a large amount of registered users decide to publish the information about paths they went through during their biking sessions. That said, the system must support a large number of registered users, publishing a vast amount of data, and it must process all the queries from users concerning paths status and obstacles. Given the considerable amount of users and data, the system must be able to easily scale-up while ensuring maximum performance. Another key aspect of BBP is availability: the system must ensure that the data is easily accessible to users and that the requested information is quickly available, all the time. Finally, the system must be capable of processing a large amount of data every time a user compiles a report of the biking trip he cycled.

3.4 Design Constraints

3.4.1 Standards Compliance

Conformity Standards

- GDPR: The system manages users' personal and administrative data, thus it must be compliant

with the General Data Protection Regulation according to the European guidelines.

- ISO/IEC 27001 - Information Security Standard: The system must be characterized by a structured and comprehensive information security management, in order to protect sensitive information and be adaptable to new risks.

Development Standards

- ISO/IEC 12207: The system must be compliant to the software lifecycle management standard by applying the best practices in all its main processes.

3.5 Software Constraints

Users access the BBP system via a mobile application, available both for Android and iOS, therefore the software constraints are relative to the mobile device used to access the platform:

- For Android: the minimum supported version is Android 10 (API 29) for security reasons, as the application works with sensitive data, such as users' location and personal information.
- For iOS: the minimum supported version is iOS 16, according to average iOS requirements.

3.5.1 Hardware Constraints

Users access the BBP system via a mobile application, available both for Android and iOS, therefore the hardware constraints are relative to the mobile device used to access the platform:

- The device used to access the platform must have a powerful enough processor in order to smoothly visualize GPU intensive data, such as maps. To this end, devices must have:
  - Android: a quad-core ARMv7 or RM64 CPU architecture, with a clock speed of at least 1.5GHz, and an absolute minimum of 2GB of RAM. Also the device must support OpenGL ES 2.0 or higher, for rendering 3D graphics.
  - iOS: at least an A12 Bionic processor and 3GB of RAM.
- The device used to access the platform must have built-in sensors, such as gyroscope, accelerometer and GPS, in order to gather all the required information about the path and other metrics.
- The device used to access the platform must have a stable internet connection, so the user can retrieve his personal activities, planned trips and statistics. Also an internet connection is crucial for searching routes between the departure and destination points, and the status of each street. Furthermore, the internet connection is necessary to retrieve community shared obstacles and add new obstacles in the report review phase.

3.6 Software System Attributes

3.6.1 Reliability

The BBP system must preserve a large amount of data retrieved from registered users, thus it should replicate data and ensure its consistency across multiple nodes. Despite the higher financial cost, this solution prevents data loss and enables data recovery. Also, the system periodically runs checksum

verification in order to validate stored data against possible corruptions. Furthermore, the system must have a secure multi-factor authentication backdoor access for administrators only, in order to ensure emergency maintenance. Note that, the usage of the backdoor must be monitored and every action reported into a log, so maximum security is guaranteed.

3.6.2 Availability

The BBP system must guarantee a highly available service, quickly accessible through users' mobile devices. The design of the system must ensure 99.9% (3-nines) of uptime, allowing only a maximum of 9 hours of downtime a year. As already said in the previous section, the system should be partitioned into multiple nodes, to ensure reliability as well as redundancy of the data. In case of node failure, the system must automatically detect it and redirect traffic to running healthy replicas, in order to maintain service continuity. This partition also allows geographical redundancy, for data replication across different physical regions to protect it against localized data centre outages. Finally, the system must implement circuit breakers between components into its design to prevent cascading failures.

3.6.3 Security

The BBP system allows users to sign in and sign up to the platform via email and password. This represents a highly sensitive part of the system, requiring some extra considerations. The password is salted, peppered and hashed with advanced encryption algorithms like *Argon2id*. The salt is randomly generated for every user at registration, then hashed and stored in the database with the encrypted password. The pepper is the same for all users and stored in a secure storage on the users' device. In order to transfer the hashed password and the hashed salt to the back-end in a secure manner, the data is encrypted and transmitted via HTTPS using the TLS 1.3 protocol. Also, the report data sent to the server and the client requested information must be transferred via the HTTPS protocol, to ensure security and consistency.

3.6.4 Maintainability

The client side of the BBP system consists of a mobile application, so maintainability is crucial. To ensure this software attribute, the mobile application is designed using the Model-View-ViewModel (MVVM) pattern, which makes it easier to separate the UI layer, from the business logic and data layer. Also, the MVVM pattern ensures Unidirectional Data Flow (UDF), decoupling the components that display state in the UI, from the ones that store and modify the state. Also, the application is subdivided into modules, so it is optimized for scalability and maintainability. All the code must be well documented and tested, to maintain a high quality standard.

The server side of the BBP system must be highly modular and ideally adopt a micro-service architecture, so it is more maintainable, testable and scalable. Every component of the server must have a clearly defined API, so it is easier for developers to integrate new components into the system.

3.6.5 Portability

The client side of the BBP system consists of a mobile application, from which users can access the platform. For this reason, the application is compatible with both Android and iOS operating systems (as specified in 3.5). The application UI must adapt to different aspect ratios, resolutions and orientations, and also different screen sizes, ranging from 4.7 inches to 12.9 inches. All the users' data is stored and synchronized in the back-end, so it's possible for users to switch from one device

to another with ease. Also, users can request to export their data in a standard JSON format so they can move it to a different service.

4 Formal Analysis

4.1 Signatures

The domain is modelled using a state-based approach that explicitly captures the evolution of the system over time. To this purpose, the Alloy library `util/ordering[State]` is imported, which introduces a total ordering over the `State` signature. This allows each system configuration to be represented as a distinct state in time and provides built-in relations such as `first`, `last`, `next`, and `prev`, enabling the specification of temporal constraints and irreversible behaviours (e.g., terminal states).

```
// Allows to define the concept of "evolution in time",
// in particular we have an entity State that represent the state
// of the system in a certain instant.
// With this module (util/..) Alloy provides automatically some features like
// first, last, next and prev which are relations of State automatically
// generated that represent the first, last, next and previous state.
open util/ordering[State]

// State is an entity that represents the state of the system
sig State {
    // set of all available paths (= inventory)
    paths: set BikePath,

    // status of a certain path (BikePath = inventory + deleted paths),
    pathStatus: BikePath -> one PathStatus,

    // set of all planned trips
    planned: RegisteredUser -> set Trip,

    // trips already performed by the user
    recorded: RegisteredUser -> set Trip,

    // set of trips completed by the user
    // but deleted from the history (history = recorded - deleted)
    deleted:  RegisteredUser -> set Trip,

    trackingEnabled: set RegisteredUser,

    reports: set Report,
    producedBy: reports -> one RegisteredUser,
    aboutTrip: reports -> one Trip,
    status: reports -> one ReportStatus,
    validatedBy: AutoReport -> lone RegisteredUser,

    tripWeather: Trip -> lone WeatherInfo,

    obstaclesOnPath: BikePath -> set Obstacle,

    alertsSent: RegisteredUser -> set Alert,

    gpsEnabled: set RegisteredUser,
    connected: set User,
    mapServiceUp: one Bool,
    weatherServiceUp: one Bool
}
```

Listing 1: State Signature.

Bike paths are modelled with an explicit status (`Active`, `Degraded`, `Unavailable`, `Deleted`) and a clear separation between the inventory of available paths and deleted ones. A key invariant enforces that only non-deleted paths appear in the system inventory, while deleted paths are terminal: once a path is marked as `Deleted`, it remains deleted in all future states and can never be reintroduced. This reflects the assumption that path removals are permanent and irreversible.

```
sig BikePath {}

enum PathStatus { Active, Degraded, Unavailable, Deleted }
```

Listing 2: Bike Path Signature.

Trips are modelled from the perspective of registered users and are divided into planned, recorded, and deleted trips. Strong constraints ensure that these categories are mutually exclusive: a trip cannot be simultaneously planned and recorded, nor planned and deleted. Each planned trip has at most one owner, preventing ambiguous ownership. Deletions are also terminal: once a trip is deleted, it remains deleted in all future states and is excluded from the user's visible history. This allows the model to faithfully represent the lifecycle of a trip from planning, to completion, and possible removal from the user's statistics.

```
sig Trip {
    path: one BikePath
}
```

Listing 3: Trip Signature.

Reports are distinguished into manual and automatic, each governed by a specific workflow. All reports are well-formed: they are produced by exactly one registered user, refer to exactly one recorded trip (completed trip), and have a single status. Automatic reports must be validated by exactly one registered user before they can be published, enforcing the goal that automatically collected information is reviewed prior to public release. Both discarded and published reports are terminal states, meaning their status cannot change in future states. Manual reports, instead, bypass validation and can only move from draft to published or discarded, reflecting their different creation process.

```
abstract sig Report {}
sig ManualReport extends Report {}
sig AutoReport extends Report {}

enum ReportStatus { Draft, Validated, Published, Discarded }
```

Listing 4: Report Signature.

Alerts represent system-driven notifications related to planned trips. Each alert is associated with exactly one planned trip and can only be sent to the registered user who owns that trip. Specialized alerts (weather or obstacle alerts) are further linked to valid `WeatherInfo` or `Obstacle` entities. This ensures that notifications are always meaningful, contextualized, and consistent with the current planning state of the user.

```
abstract sig Alert {
    plan: one Trip
}


```

```
sig WeatherAlert extends Alert {
    info: one WeatherInfo
}

sig ObstacleAlert extends Alert {
    obstacle: one Obstacle
}
```

Listing 5: Alert Signature.

Overall, the choice to model the domain as a sequence of ordered states enables the precise expression of temporal and lifecycle constraints that are central to the system's behaviour. Irreversible actions (such as deletions, publications, or discards), ownership constraints, and validation workflows are naturally captured through state evolution, resulting in a coherent and expressive formalization of the domain.

4.2 Functions, Predicates and Assertions

The introduced function `nonDeletedPaths` explicitly defines the set of bike paths that are not marked as `Deleted` in a given system state. The assertion `G6_InventoryEqualsAllNonDeletedPaths` verifies that, for every state of the system, the inventory of available bike paths exactly corresponds to this set. The absence of counterexamples in the Alloy check provides evidence that the modelled inventory is consistent with the intended semantics: deleted paths are never exposed to users, while all non-deleted paths are always included in the inventory. Consequently, the assertion supports Goal G6 by ensuring that accessible bike path information is correctly derived from the path status and remains coherent across state evolutions.

```
// Return the set of non-deleted paths in a given state s
fun nonDeletedPaths[s: State]: set BikePath {
    { p: BikePath | s.pathStatus[p] != Deleted }
}

// Check if inventory in state s is the same as the set of non-deleted paths
// in a given state s (retuned by the function)
assert G6_InventoryEqualsAllNonDeletedPaths {
    all s: State | s.paths = nonDeletedPaths[s]
}

check G6_InventoryEqualsAllNonDeletedPaths for
      3 State,
      1 RegisteredUser,
      1 UnregisteredUser,
      7 BikePath,
      2 Point,
      6 Trip,
      0 Report,
      0 WeatherInfo,
      0 Obstacle,
      0 Alert
```

Listing 6: Assertion for Goal G6.

The function `visibleHistory` defines the set of trips that are effectively visible to a registered user by excluding deleted trips from the set of recorded ones. The assertion `G1_VisibleHistoryWellFormed` verifies two key properties: first, that deleted trips never appear in the user's visible history, and second, that the visible history contains only completed trips, i.e.

it is always a subset of the recorded trips. The Alloy check produced no counterexamples within the given scope, confirming that the modelled history is consistent and well-formed in every system state. This result supports Goal G1 by ensuring that users can reliably track their past cycling activities without inconsistencies caused by deleted records.

```
fun visibleHistory[s: State, u: RegisteredUser]: set Trip {
    s.recorded[u] - s.deleted[u]
}

assert G1_VisibleHistoryWellFormed {
    all s: State, u: RegisteredUser | {
        // no "deleted trips" appear in the visible history
        // it's trivial because we have defined visibleHistory as
        // the difference, but we need to show the consistency of
        // the RU's history
        no (visibleHistory[s,u] & s.deleted[u])

        // visible history contains only completed trips
        // (i.e., subset of recorded)
        visibleHistory[s,u] in s.recorded[u]
    }
}

check G1_VisibleHistoryWellFormed for
      3 State,
      2 User,
      2 RegisteredUser,
      0 UnregisteredUser,
      4 Trip,
      2 BikePath,
      2 Point,
      0 Report,
      0 Alert,
      0 WeatherInfo,
      0 Obstacle
```

Listing 7: Assertion for Goal G1.

The assertion `G3_AutoReportsValidatedBeforePublication` formalizes Goal G3 by enforcing three critical constraints on the lifecycle of reports. First, it ensures that every automatically generated report that reaches the Published state has been validated by exactly one registered user. Second, it guarantees that once an automatic report is marked as Discarded, its status remains immutable in all future system states. Third, it enforces immutability for all published reports—both manual and automatic—preventing any further state changes after publication. The absence of counterexamples gives us confidence about the fact that all three properties consistently hold across every possible execution trace.

```
assert G3_AutoReportsValidatedBeforePublication {
    all s: State | {
        // (1) If an automatic report is Published,
        // it must have exactly one RegisteredUser validator
        all r: AutoReport & s.reports | (s.status[r] = Published)
        implies (one s.validatedBy[r])

        // (2) If an automatic report is Discarded,
        // it never changes status in any future state
        all r: AutoReport & s.reports | (s.status[r] = Discarded)
        implies all s2: s.*ordering/next | s2.status[r] = Discarded
```

```
12
13          // (3) If a report (manual or automatic) is Published,
14          // it never changes status in any future state
15          all r: s.reports | (s.status[r] = Published)
16          implies all s2: s.*ordering/next | s2.status[r] = Published
17      }
18  }
19
20  check G3_AutoReportsValidatedBeforePublication for 6 but 8 State
```

Listing 8: Assertion for Goal G3.

The predicate `showPlanningWeatherAlertAndCompletion` is used to illustrate, through a concrete execution trace, the lifecycle of a biking trip from planning to completion, including the interaction with the weather alert mechanism. By exploiting the `util/ordering[State]` module, the predicate explicitly constrains four consecutive system states (s_0, s_1, s_2, s_3), allowing the evolution of the system to be observed step by step.

```
1   pred showPlanningWeatherAlertAndCompletion {
2       some u: RegisteredUser, t: Trip, a: WeatherAlert,
3       s0, s1, s2, s3: State | {
4           // consecutive states
5           s1 = s0.next
6           s2 = s1.next
7           s3 = s2.next
8
9           // (1) planning: t is added to planned trips of u
10          t not in s0.planned[u]
11          t in     s1.planned[u]
12
13          // (2) weather alert: alert sent to u about the planned trip t
14          a not in s1.alertsSent[u]
15          a in     s2.alertsSent[u]
16          a.plan = t
17          some a.info
18          s2.weatherServiceUp = True
19
20          // ensure it is indeed a planned trip at alert time
21          // (coherent with AlertsWellFormed)
22          t in s2.planned[u]
23
24          // (3) completion: trip becomes recorded (and is no longer planned)
25          t in     s2.planned[u]
26          t not in s2.recorded[u]
27
28          t not in s3.planned[u]
29          t in     s3.recorded[u]
30      }
31  }
32
33  run showPlanningWeatherAlertAndCompletion for 1 but 4 State
```

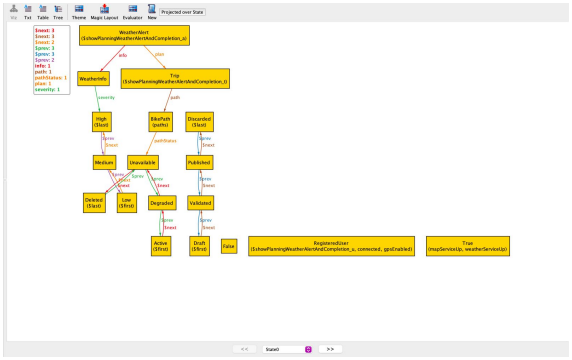
Listing 9: Predicate showing Trip Lifecycle.

In the first transition $(s_0 \rightarrow s_1)$, the predicate models the planning phase: a trip `t`, initially absent from the planned trips of a registered user `u`, is added to `u`'s planned set. This transition represents the moment in which the user schedules a trip, making it an explicit system-managed entity. The visualization clearly shows the appearance of the planned relationship between the

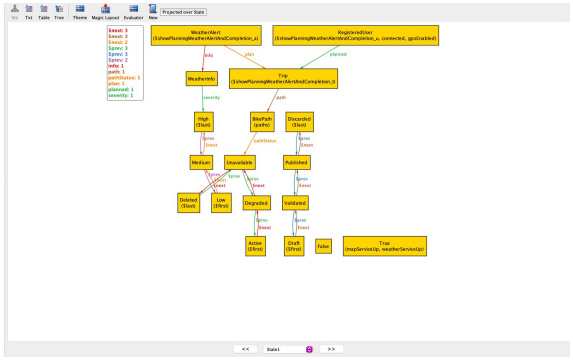
`RegisteredUser` and the `Trip` in `State1`, marking the successful completion of the planning action.

In the second transition ($s_1 \rightarrow s_2$), the predicate captures the system's reactive behaviour: once the trip is planned and the weather service is available, a `WeatherAlert` is generated and sent to the same user. The alert is explicitly linked to the planned trip `t` via the plan relation and enriched with weather information (`a.info`). This step highlights how planned trips enable proactive system behaviour, such as monitoring external conditions and notifying users of relevant issues.

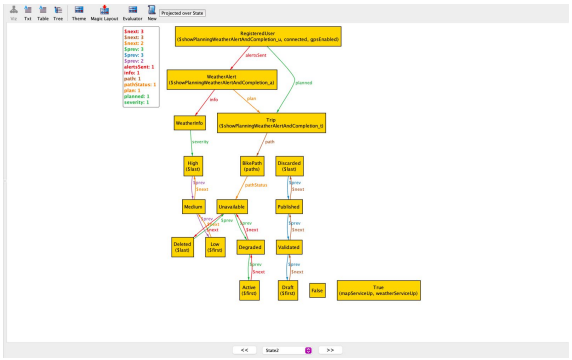
Finally, in the last transition ($s_2 \rightarrow s_3$), the predicate models trip completion. The trip `t` is removed from the user's planned trips and added to the recorded set, representing a completed cycling activity. This enforces the conceptual separation between planned and completed trips and demonstrates how the system updates the user's history once the trip is performed. Overall, the generated instance visually confirms the correctness of the modelled process: a trip is first planned, then monitored through alerts while planned, and finally recorded upon completion. The predicate therefore provides a clear and concrete validation of the planning workflow and its integration with alerting and trip lifecycle management within the system.



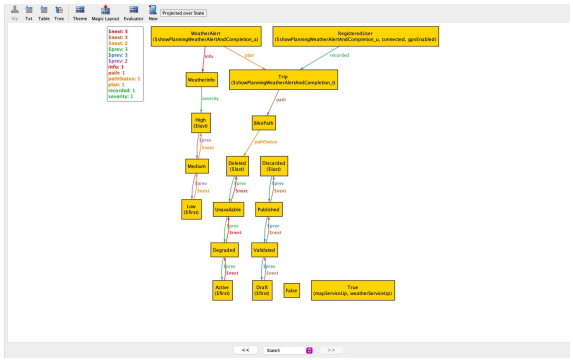
(a) State 0.



(b) State 1.



(c) State 2.



(d) State 3.

Figure 21: Alloy States Visualization.

4.3 Complete Alloy Formalization

```
// Allows to define the concept of "evolution in time",
// in particular we have an entity State that represent the state
// of the system in a certain instant.
open util/ordering[State]

abstract sig Report {}
sig ManualReport extends Report {}
sig AutoReport extends Report {}

enum ReportStatus { Draft, Validated, Published, Discarded }
enum Severity { Low, Medium, High }
enum PathStatus { Active, Degraded, Unavailable, Deleted }

sig WeatherInfo {
    severity: one Severity
}

sig Obstacle {
    severity: one Severity
}

abstract sig Alert {
    plan: one Trip
}

sig WeatherAlert extends Alert {
    info: one WeatherInfo
}

sig ObstacleAlert extends Alert {
    obstacle: one Obstacle
}

// True or False values in order to represent the fact that the map service
// or the weather service is available or not
abstract sig Bool {}
one sig True extends Bool {}
one sig False extends Bool {}

// State is an entity that represents the state of the system
sig State {
    // set of all available paths (= inventory)
    paths: set BikePath,

    // status of a certain path (BikePath = inventory + deleted paths),
    pathStatus: BikePath -> one PathStatus,

    // set of all planned trips
    planned: RegisteredUser -> set Trip,

    // trips already performed by the user
    recorded: RegisteredUser -> set Trip,

    // set of trips completed by the user
    // but deleted from the history (history = recorded - deleted)
    deleted:  RegisteredUser -> set Trip,

    trackingEnabled: set RegisteredUser,
```

```
59
60      reports: set Report,
61      producedBy: reports -> one RegisteredUser,
62      aboutTrip: reports -> one Trip,
63      status: reports -> one ReportStatus,
64      validatedBy: AutoReport -> lone RegisteredUser,
65
66      tripWeather: Trip -> lone WeatherInfo,
67      obstaclesOnPath: BikePath -> set Obstacle,
68      alertsSent: RegisteredUser -> set Alert,
69
70      gpsEnabled: set RegisteredUser,
71      connected: set User,
72      mapServiceUp: one Bool,
73      weatherServiceUp: one Bool
74  }
75
76  fact PartitionUsers {
77      // 2 entities that extend the same abstract entity
78      // are by default disjointed
79      User = RegisteredUser + UnregisteredUser
80  }
81
82  fact DomainAssumptions {
83      all s: State | {
84          s.gpsEnabled = RegisteredUser
85          s.connected = User
86          s.mapServiceUp = True
87          s.weatherServiceUp = True
88      }
89  }
90
91  fact InventoryShowsOnlyNonDeletedPaths {
92      all s: State | all p: BikePath | (p in s.paths) iff
93      (s.pathStatus[p] != Deleted)
94  }
95
96  // Once a path is deleted it cannot be re-introduced in the inventory
97  fact DeletedPathIsTerminal {
98      all s: State | all p: BikePath | (s.pathStatus[p] = Deleted) implies
99      all s2: s.*ordering/next | s2.pathStatus[p] = Deleted
100 }
101
102 // Once a trip is deleted, it will stay deleted forever
103 // (so it will not be shown in the history of the RU)
104 fact DeletedTripsAreTerminal {
105     all s: State, s2: s.*ordering/next | s.deleted in s2.deleted
106 }
107
108 // The history of the RU shows only non deleted trips
109 fact DeletedTripsNotInHistory {
110     all s: State, u: RegisteredUser | no (s.recorded[u] & s.deleted[u])
111 }
112
113 fact ReportsWellFormed {
114     all s: State | {
115         // Each report is produced by exactly one RU
116         all r: s.reports | one s.producedBy[r]
117         // Each report refers to exactly one trip
118         all r: s.reports | one s.aboutTrip[r]
```

```
        // Each report has exactly one status
        all r: s.reports | one s.status[r]
        // The report must refer to a recorded trip of the producing user
        all r: s.reports | s.aboutTrip[r] in s.recorded[s.producedBy[r]]
    }
}

fact AutoReportValidationWorkflow {
    all s: State | all r: AutoReport & s.reports | {
        // If the report is validated or published
        // it must have exactly one validator
        (s.status[r] in Validated + Published) implies one s.validatedBy[r]
    }

    // If the report is discarded then it must stay discarded forever
    all s: State | all r: AutoReport & s.reports | (s.status[r] = Discarded)
    implies all s2: s.*ordering/next | s2.status[r] = Discarded

    all s: State | all r: AutoReport & s.reports | (s.status[r] = Published)
    implies all s2: s.*ordering/next | s2.status[r] = Published
}

fact ManualReportWorkflow {
    // For all the manual reports in the current state,
    // if they have a different status then they are validated
    // (so draft or published)
    all s: State | all r: ManualReport & s.reports | s.status[r]
    not in Validated

    all s: State | all r: ManualReport & s.reports |
    (s.status[r] = Published) implies
    all s2: s.*ordering/next | s2.status[r] = Published

    all s: State | all r: ManualReport & s.reports |
    (s.status[r] = Discarded) implies
    all s2: s.*ordering/next | s2.status[r] = Discarded
}

fact PublicationRequiresExistence {
    // If the report is published then it must exists in the current state
    all s: State | all r: s.reports | (s.status[r] = Published)
    implies (r in s.reports)
}

fact TripWeatherOnlyForTripsInTheInventory {
    // If at least one trip has available weather info...
    all s: State | all t: Trip | some s.tripWeather[t]
    // ... then the trip is associated with a path in the inventory
    // and the external weather service is available
    implies (t.path in s.paths and s.weatherServiceUp = True)
}

// A trip can only be planned or recorded (once performed become recorded)
fact PlannedAndRecordedAreDisjoint {
    all s: State, u: RegisteredUser | no (s.planned[u] & s.recorded[u])
    all s: State, u: RegisteredUser | no (s.planned[u] & s.deleted[u])
}

fact PlannedTripHasSingleOwner {
    all s: State, t: Trip | lone (s.planned.t)
```

```
}

// An alert can be sent only to a RU
// and must refer to one of that user's planned trips
fact AlertsWellFormed {
    all s: State, u: RegisteredUser, a: Alert | (a in s.alertsSent[u])
    implies (a.plan in s.planned[u])
}

// Return the set of non-deleted paths in a given state s
fun nonDeletedPaths[s: State]: set BikePath {
    { p: BikePath | s.pathStatus[p] != Deleted }
}

// Check if inventory in state s is the same as the set of non-deleted paths
// in a given state s (retuned by the function)
assert G6_InventoryEqualsAllNonDeletedPaths {
    all s: State | s.paths = nonDeletedPaths[s]
}

check G6_InventoryEqualsAllNonDeletedPaths for
      3 State,
      1 RegisteredUser,
      1 UnregisteredUser,
      7 BikePath,
      2 Point,
      6 Trip,
      0 Report,
      0 WeatherInfo,
      0 Obstacle,
      0 Alert

      fun visibleHistory[s: State, u: RegisteredUser]: set Trip {
    s.recorded[u] - s.deleted[u]
}

assert G1_VisibleHistoryWellFormed {
    all s: State, u: RegisteredUser | {
        // no "deleted trips" appear in the visible history
        // it's trivial because we have defined visibleHistory as
        // the difference, but we need to show the consistency of
        // the RU's history
        no (visibleHistory[s,u] & s.deleted[u])

        // visible history contains only completed trips (i.e., subset of
    recorded)
        visibleHistory[s,u] in s.recorded[u]
    }
}

check G1_VisibleHistoryWellFormed for
      3 State,
      2 User,
      2 RegisteredUser,
      0 UnregisteredUser,
      4 Trip,
      2 BikePath,
      2 Point,
      0 Report,
      0 Alert,
```

```
      0 WeatherInfo,
      0 Obstacle

assert G3_AutoReportsValidatedBeforePublication {
    all s: State | {
        // (1) If an automatic report is Published,
        // it must have exactly one RegisteredUser validator
        all r: AutoReport & s.reports | (s.status[r] = Published)
        implies (one s.validatedBy[r])

        // (2) If an automatic report is Discarded,
        // it never changes status in any future state
        all r: AutoReport & s.reports | (s.status[r] = Discarded)
        implies all s2: s.*ordering/next | s2.status[r] = Discarded

        // (3) If a report (manual or automatic) is Published,
        // it never changes status in any future state
        all r: s.reports | (s.status[r] = Published)
        implies all s2: s.*ordering/next | s2.status[r] = Published
    }
}

check G3_AutoReportsValidatedBeforePublication for 6 but 8 State

pred showPlanningWeatherAlertAndCompletion {
    some u: RegisteredUser, t: Trip, a: WeatherAlert,
    s0, s1, s2, s3: State | {
        // consecutive states
        s1 = s0.next
        s2 = s1.next
        s3 = s2.next

        // (1) planning: t is added to planned trips of u
        t not in s0.planned[u]
        t in     s1.planned[u]

        // (2) weather alert: alert sent to u about the planned trip t
        a not in s1.alertsSent[u]
        a in     s2.alertsSent[u]
        a.plan = t
        some a.info
        s2.weatherServiceUp = True

        // ensure it is indeed a planned trip at alert time
        // (coherent with AlertsWellFormed)
        t in s2.planned[u]

        // (3) completion: trip becomes recorded (and is no longer planned)
        t in     s2.planned[u]
        t not in s2.recorded[u]

        t not in s3.planned[u]
        t in     s3.recorded[u]
    }
}

run showPlanningWeatherAlertAndCompletion for 1 but 4 State
```

Listing 10: Complete Alloy Formalization.

5 Effort Spent

The following table displays the effort spent in hours of both group members on the sections of the document. The division is only approximative and each section still required the collaboration of both members.

	Section 1	**Section 2**	**Section 3**	**Section 4**
Vokrri Fabio	6	4	35	5
Soufien Sadgal	8	5	25	15

Table 20: Effort spent in hours.

6 References